REQUI TRADING · ENGINEERING

Autonomous Execution Development Guide

Version 2.2 — auto-execution with per-user confirmation choice, order history, positions & P&L, and strategy self-identification

Document type: Engineering handoff — full source, diffs, and rollout steps

Supersedes: Version 2.1 (Intelligence & System Check guide)

Date: August 4, 2026 · Classification: Confidential — internal engineering only

Contents

1 What this version ships

Version 2.2 turns the Autonomous module from a **signals-and-tickets** system into a **live execution** system. In v2.1 every order — no matter where it came from — stopped at the confirmation gate and waited for a human to type `CONFIRM ORDER [TICKET_ID]`. In v2.2 the confirmation gate is still there, but it is now **a per-user choice**: each user decides whether Autonomous engine proposals execute automatically or wait for confirmation. Four capabilities ship together:

1. **Auto-execution (user-selectable).** A new AUTO-EXECUTE toggle on the Autonomous page. When ON, confirmed setups from the autonomous engine execute immediately — no confirmation string. When OFF (the default), behaviour is identical to v2.1. The toggle applies *only* to proposals originated by the Autonomous engine; trades typed into Intelligence chat and manual orders always require confirmation.
2. **Order history.** Every ticket that reached the broker is listed in a new Orders & P&L panel — symbol, side, quantity, fill price, state, timestamp — with an AUTO badge on executions that happened without human confirmation.
3. **Positions & P&L.** A new positions ledger books fills into open positions (average-in on buys), closes them on sells, and computes realized P&L. Open positions are marked to the live data feed for unrealized P&L, with an honest “no mark” state when the feed has no price.
4. **Strategy self-identification.** The engine reads each symbol’s tape — gap, position vs VWAP, RSI, relative volume — names the best-fit strategy itself, and reports a confidence score with an auditable reason list. Available as an “AUTO” option in the strategy dropdown and on every scanner candidate.

> **READ THIS FIRST**
>
> Auto-execution is **off by default** and **fails closed**: any error reading the user’s preference resolves to OFF. Nothing in this version loosens risk limits, sizing caps, governance checks, or the ticket state machine — auto-executed orders travel the exact same broker path, with the same idempotency key and the same dead-end-on-UNKNOWN rule, as human-confirmed orders. The audit trail for auto-executions is deliberately *louder* than for manual ones (a CRITICAL-priority alert on every fill).

2 What changed vs Version 2.1

Table 1 — Delta summary, v2.1 → v2.2

Area	v2.1 behaviour	v2.2 behaviour
Order execution	All orders blocked at the confirmation gate; human must type CONFIRM ORDER.	Per-user AUTO-EXECUTE toggle. ON = autonomous proposals execute immediately; OFF = unchanged. Chat/manual trades still always require confirmation.
Order history	None — tickets visible only as transient alerts.	Persistent order history table in the UI, backed by the tickets table, with AUTO badge on auto-executed rows.
P&L	None — fills were not booked anywhere.	New `positions` ledger. Realized P&L on closes, unrealized P&L marked to the live feed, summary cards (realized / unrealized / open / mode).
Strategy selection	Human picks the strategy config per monitor.	Engine can self-identify from the tape (gap / VWAP / RSI / RVOL) with confidence + reasons; “AUTO” option in the monitor dropdown and scanner.
Scanner candidates	Suggested strategy label only.	Label plus confidence badge (green ≥ 70, amber ≥ 45) driven by the same identifier.
Ticket record	—	New `autoExecuted` flag and `confirmedAt` stamp on auto-executions.
User record	—	New `autoExecute` preference column, default false.

Unchanged: the ticket state machine, the confirmation gate for INTELLIGENCE and MANUAL origins, sizing (`sizePosition`), governance package checks, the learning loop, conversation memory, and the System Check module. All v2.1 components pass the same System Check probes in v2.2.

3 Auto-execution — with or without human confirmation

3.1 User-facing behaviour

The Autonomous page gains an **Orders & P&L** panel whose header carries the AUTO-EXECUTE toggle. The toggle is a plain ON/OFF switch persisted per user:

- **OFF (default).** A confirmed setup produces a ticket in `READY_FOR_CONFIRMATION` and an alert. The order routes only after the user types `CONFIRM ORDER [TICKET_ID]` — exactly the v2.1 flow.
- **ON.** The moment a monitor's state machine reaches CONFIRMED and a ticket is proposed, the backend calls `autoExecuteTicket` instead of waiting. The ticket transitions CONFIRMED → SUBMITTING → FILLED/WORKING with no human in the loop, and the user sees the result in order history, positions, and a CRITICAL alert. An amber banner stays visible whenever the mode is ON: "Autonomous engine proposals will buy without asking for your CONFIRM string… Natural-language (chat) trades still always require confirmation."

3.2 Backend design

Auto-execution is a **mirror** of the existing confirmation path, not a new path. `autoExecuteTicket` (Appendix E) is `confirmTicket` minus the confirmation-string check. It reuses everything else: the same ticket artifact, the same broker adapter resolution, the same idempotency key (`clientOrderId = ticket.idempotencyKey`), the same UNKNOWN-is-a-dead-end rule (no resubmission, ever), and the same terminal states. The differences are exactly three:

1. The transition to CONFIRMED stamps `autoExecuted: true` on the ticket row.
2. On fill, the alert is priority CRITICAL and titled "AUTO-EXECUTED — Order Filled/Working", with a body that records "AUTO-EXECUTE was ON · no human confirmation (user setting)".
3. On fill, the position ledger is updated via `recordFill` (Appendix A) — the same hook the manual confirmation path now uses.

3.3 Scope — who is allowed to skip confirmation

Only tickets with `origin: "AUTONOMOUS"` ever reach `autoExecuteTicket`, because the only caller is the monitor loop (`api/engine/monitor.ts`, Appendix F). INTELLIGENCE (chat) and MANUAL tickets have no code path to it — the UI never calls it, the router never exposes it, and the natural-language trade tool still demands the CONFIRM string. This is enforced structurally (no route), not by a runtime check that could be bypassed.

3.4 Failure behaviour

- **Preference read fails** → treated as OFF (`isAutoExecuteEnabled` catches and returns false).
- **Auto-execution fails mid-flight** (broker unreachable, rejected, window expired) → the ticket remains staged; the monitor's event detail says the ticket "remains staged for manual CONFIRM", so the user can still execute it by hand or reject it.
- **Toggle flipped mid-day** → takes effect on the next proposal; already-staged tickets are unaffected and still confirmable by hand.

FRONTEND

The toggle, banner, and mode indicator live in `src/components/OrdersPanel.tsx` (Appendix B), backed by `engine.getAutoExecute` / `engine.setAutoExecute` (Appendix H). The live-monitors header in `EnginePanel.tsx` now reads “confirmation-gated unless AUTO-EXECUTE is ON” (Appendix I).

4 Order history

Order history is a query over the existing `order_tickets` table — no new storage was needed, because every order (confirmed or auto-executed) is already a ticket row with full lifecycle fields. `listOrders(userId, limit)` (Appendix A) returns the most recent tickets that actually left the staging area, i.e. any state other than `CREATED`, newest first. Each row carries: ticket id, symbol, side, quantity, limit/stop prices, current state, the `autoExecuted` flag, broker order id, filled quantity, average fill price, and timestamps.

The UI renders this as the lower table of the Orders & P&L panel (Appendix B) with a 15-second refresh. Auto-executed rows display an **AUTO** badge next to the state, so a glance separates machine-executed orders from human-confirmed ones — this is the day-to-day audit surface that backs the louder alert trail described in Section 3.

5 Positions and P&L

5.1 The positions ledger

A new table, `positions` (Appendix C), is the book of record for P&L. One row is one open (or closed) position in one symbol for one user, sourced from a specific fill ticket (`sourceTicketId`). The bookkeeping rules in `recordFill` (Appendix A):

- **BUY with no open position** → insert a new OPEN row: quantity, average entry = fill price.
- **BUY with an open position** → average in: new average entry is the quantity-weighted mean of the existing entry and the new fill.
- **SELL** → close the oldest open position. Realized P&L = (fill price − average entry) × closing quantity, booked on the row with exit price and close timestamp. A partial sell splits the row: the closed piece books realized P&L, the remainder stays OPEN with the original average entry.

Both fill paths — human-confirmed and auto-executed — call the same hook, so the ledger is complete regardless of execution mode. The hook is wrapped in `.catch()` at the call site: a bookkeeping failure can never break an execution that already happened at the broker (reconciliation is a separate operational concern, and the CRITICAL alert preserves the fill facts).

5.2 Unrealized P&L and marks

`listPositions` marks each open position against the live market-data feed's last price. If the feed has no price for the symbol (market closed and no cached feed, symbol untracked), the row reports **no mark** and its unrealized P&L is `null` — never a fabricated number. `getPnlSummary` aggregates: realized total, unrealized total (marked positions only), open/closed counts, win/loss counts, and the count of unmarked positions, plus a human-readable note when marks are missing.

5.3 UI

The top of the Orders & P&L panel shows four cards — Realized P&L, Unrealized P&L, Open positions, and Execution mode (AUTO/CONFIRM) — followed by the open-positions table (symbol, qty, average entry, mark, unrealized P&L with up/down colouring, source ticket). Both tables refresh every 15 seconds via tRPC invalidation intervals.

6 Strategy self-identification

6.1 What it does

`identifyStrategy` (Appendix G) classifies a symbol's current tape into the best-fit built-in strategy config and — as important as the pick itself — explains it. Input features: opening gap %, position vs VWAP %, RSI-14, relative volume. Output:

- `strategyId` — one of the existing governed configs (no new strategies were added);
- `confidence` — 0–100, clamped to 5–95 so the engine never claims certainty;
- `reasons` — an auditable list of every evidence line that moved the score, each carrying the actual numbers ("RSI 31 oversold — reversal setup strengthens").

6.2 Classification logic

Table 2 — Identification rules (all thresholds from the existing engine constants)

Identified strategy	Signature	Confidence adjustments
bottom_fish_reversal	Down-gap beyond the scanner threshold *and* trading below VWAP — capitulation profile.	+20 RSI < 35 (oversold); +10 RVOL ≥ threshold; −15 RSI > 50 (evidence weakens).
post_earnings_continuation	Up-gap beyond threshold, or above VWAP with elevated RVOL — event-strength profile.	+15 RSI in 55–70 momentum band; +15 RVOL ≥ 2; −20 RSI > 75 (chase risk).
general_intraday_momentum (default)	Liquid mover without a clear gap/reversal signature.	Base 40; +10 above VWAP; +10 RSI < 30 (watch for reversal upgrade).

6.3 Where it surfaces

- **Monitor start — "AUTO".** The strategy dropdown on the Autonomous page has a new "AUTO — self-identify strategy (monitor only)" option. The router resolves it at start time against the live feed and returns the identification (id, label, confidence, reasons) with the started monitor; if the symbol has no feed data yet, the call fails with a clear message rather than guessing.
- **Scanner.** Every candidate card now shows a confidence badge next to the suggested strategy (green ≥ 70, amber 45–69, grey below) — same identifier, same numbers.
- **Ad-hoc query.** `engine.identify({ symbol })` exposes the identifier directly, and Intelligence can call it when a user asks "what strategy fits this tape?"

> **DESIGN CONSTRAINT**
>
> Self-identification only ever selects among existing, governed strategy configs — it cannot invent parameters, widen stops, or change sizing. Confidence affects *ranking and presentation*, never authority.

7 Safety and governance invariants

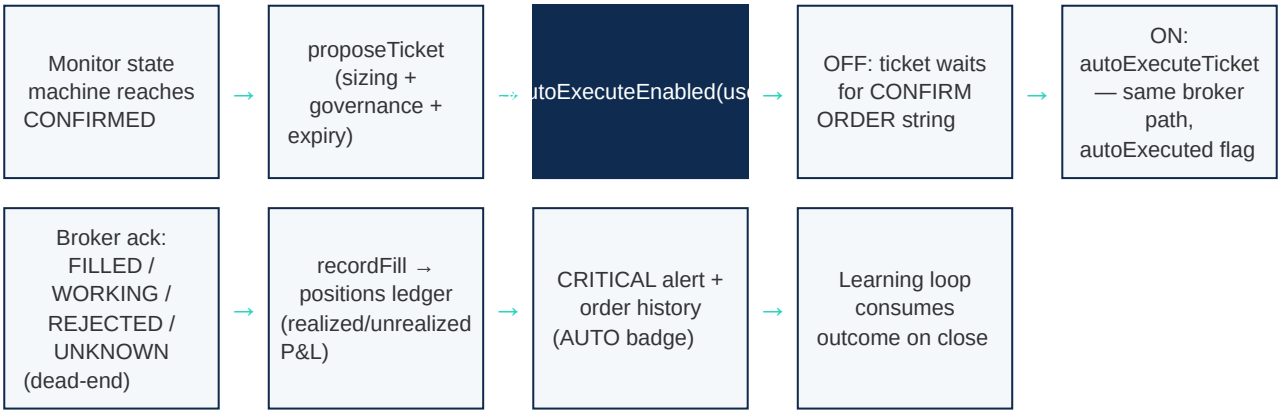
These properties held in v2.1 and are load-bearing in v2.2. Any future change that breaks one is a regression by definition:

Table 3 — Invariants and how v2.2 preserves them

Invariant	Mechanism in v2.2
Default is safe	`users.autoExecute` defaults to false at the column level; the read path fails closed to false on any error.
Scope is structural	`autoExecuteTicket` has no tRPC route and no UI caller. Only the monitor loop calls it, and monitors only propose `origin: AUTONOMOUS` tickets. Chat and manual trades physically cannot reach it.
One execution path	Auto-execute reuses the confirm path's broker submission verbatim: same adapter, same idempotency key, same UNKNOWN-dead-end (never resubmit), same terminal states.
Louder audit, not quieter	Auto-executions stamp `autoExecuted` + `confirmedAt` on the ticket, emit a CRITICAL alert recording "no human confirmation (user setting)", and carry an AUTO badge in order history.
Risk limits untouched	Sizing (`sizePosition`), risk %, position caps, and governance-package checks run before any ticket exists; nothing in the execution toggle bypasses them.
Bookkeeping never blocks execution	`recordFill` is invoked with `.catch()` after the broker ack; a P&L write failure cannot strand a live order.

8 Execution flow

End-to-end flow of one autonomous cycle in v2.2. The branch point is the user's AUTO-EXECUTE preference, evaluated after the ticket is proposed:



Both branches converge on identical downstream behaviour; the only difference is *who* moves the ticket from READY_FOR_CONFIRMATION to CONFIRMED.

9 Database migration

One new table and two new columns. Everything is handled automatically at boot by `api/lib/ensure-schema.ts` (idempotent `CREATE TABLE IF NOT EXISTS` / column-add guards) — there is no manual migration to run. After a successful boot the log line reads `schema ensured (16 tables)` (v2.1 printed 15).

9.1 New columns

- `users.autoExecute TINYINT(1) NOT NULL DEFAULT 0` — the per-user execution preference.
- `order_tickets.autoExecuted TINYINT(1) NOT NULL DEFAULT 0` — audit flag set when a ticket executes via the auto path.

9.2 New table: positions

Table 4 — positions — column reference

Column	Type	Purpose
`id`	BIGINT AUTO_INCREMENT PK	Row id.
`userId`	INT (indexed with status)	Owner; all queries are user-scoped.
`symbol`	VARCHAR(16)	Instrument.
`quantity`	INT	Open (or closed) share count.
`avgEntry`	DECIMAL(12,4)	Quantity-weighted average entry.
`sourceTicketId`	VARCHAR(64)	The fill ticket that opened the row (joins to order history).
`status`	ENUM(OPEN, CLOSED)	OPEN rows are marked for unrealized P&L.
`openedAt` / `closedAt`	TIMESTAMP	Lifecycle timestamps.
`exitPrice`	DECIMAL(12,4) NULL	Fill price of the closing sell.
`realizedPnl`	DECIMAL(12,2) NULL	(exit − avgEntry) × qty, booked on close.

> **DEPLOYMENT REMINDER (UNCHANGED FROM V2.1)**
>
> The production server only listens when started with `NODE_ENV=production node dist/boot.js` — without it the process exits silently. Delete stale `*.tsbuildinfo` before `tsc --noEmit`. If port 3000 answers with an old build, a respawned dev server is holding it — kill the vite process and start the production bundle.

10 Implementation steps

Apply in this order. Appendices carry the full source for new files and exact snippets for edits; every snippet is copied verbatim from the running build.

1. **Schema.** Apply the three edits in Appendix C to `db/schema.ts` (user column, ticket column, positions table).
2. **Boot migration.** Apply Appendix D to `api/lib/ensure-schema.ts` (CREATE TABLE, two ALTER statements, add `"positions"` to `USER_ID_TABLES`).
3. **Portfolio engine.** Create `api/engine/portfolio.ts` from Appendix A (new file).
4. **Ticket layer.** Apply Appendix E to `api/queries/tickets.ts` (import, fill hook in `confirmTicket`, new `autoExecuteTicket`).
5. **Monitor branch.** Apply Appendix F to `api/engine/monitor.ts` (imports + the ON/OFF branch after `proposeTicket`).
6. **Strategy identification.** Apply Appendix G to `api/engine/scanner.ts` (interface + `identifyStrategy`, Candidate fields, call inside `scanFeeds`).
7. **Router.** Apply Appendix H to `api/engine-router.ts` (imports, `identifyForSymbol`, AUTO branch in `monitorStart`, `identify` query, five portfolio routes).
8. **UI — new panel.** Create `src/components/OrdersPanel.tsx` from Appendix B and mount it in `src/pages/app/Autonomous.tsx` (Appendix J).
9. **UI — engine panel.** Apply Appendix I to `src/components/EnginePanel.tsx` (AUTO option, confidence badge, header copy).
10. **Verify.** `rm -f *.tsbuildinfo && npx tsc --noEmit` (must be silent), then `npm run build`, then boot with `NODE_ENV=production` and confirm `schema ensured (16 tables)` plus HTTP 401 (auth-gated) on `/api/trpc/engine.pnl`, `engine.orders`, `engine.positions`, `engine.getAutoExecute`, `engine.identify`.
11. **Exercise.** Run the test plan in Section 12 against a paper account before any user toggles AUTO-EXECUTE on with a live broker.

11 API reference — new and changed routes

Table 5 — tRPC routes added or changed in v2.2 (all user-scoped, auth required)

Route	Kind	Behaviour
`engine.getAutoExecute`	query	Returns `{ enabled }` for the current user (fails closed to false).
`engine.setAutoExecute`	mutation	Input `{ enabled: boolean }`. Persists the preference to `users.autoExecute`.
`engine.orders`	query	Latest 100 tickets past CREATED, newest first, incl. `autoExecuted`.
`engine.positions`	query	Open/closed positions with live marks and unrealized P&L (null when unmarked).
`engine.pnl`	query	Realized/unrealized totals, open/closed/win/loss counts, unmarked count + note.
`engine.identify`	query	Input `{ symbol }` → strategyId, label, confidence, reasons from the live tape.
`engine.monitorStart`	mutation	Now accepts `strategyId: "AUTO"`; resolves via the feed and returns the identification with the started monitor.
`engine.scan`	query	Candidates now carry `confidence` and `reasons`.
`engine.confirm` (chat CONFIRM path)	mutation	Unchanged semantics; on FILLED now also books the fill into the positions ledger.

`autoExecuteTicket` is deliberately **not** a route (Section 7).

12 Test plan

Run against a paper broker account. Each step names the pass condition.

1. **Default safe.** Fresh user → Orders & P&L panel shows Execution mode: CONFIRM and the toggle OFF. Pass: no banner visible.
2. **Confirm path intact.** Toggle OFF, start a monitor, Evaluate now until a ticket is staged. Pass: ticket waits; `CONFIRM ORDER [TICKET_ID]` executes; order history row has *no* AUTO badge; positions row opens; CRITICAL alert absent (normal EXECUTION alert instead).
3. **Auto path.** Toggle ON (banner appears), start/Evaluate a monitor on a tracked symbol. Pass: ticket executes without any CONFIRM string; CRITICAL "AUTO-EXECUTED" alert arrives; history row shows AUTO badge; position opens or averages in.
4. **Scope.** With the toggle still ON, ask Intelligence chat for a trade. Pass: it still produces a ticket and demands the CONFIRM string — nothing auto-executes from chat.
5. **Failure honesty.** Stop the broker gateway, Evaluate a monitor with the toggle ON. Pass: event detail says auto-execute failed and the ticket "remains staged for manual CONFIRM"; no crash, no duplicate submission after recovery.
6. **P&L math.** After a buy then a sell fill: realized P&L = (sell − avgEntry) × qty on the closed row; unrealized shows only while the feed has a mark, "no mark" otherwise.
7. **Self-identification.** Start a monitor with strategy "AUTO". Pass: response includes identified strategyId/label/confidence/reasons; scanner candidates show confidence badges; `engine.identify` on an untracked symbol fails with the "no feed data" message.
8. **Schema.** Boot log shows `schema ensured (16 tables)` on first boot and is idempotent on reboot.

Appendix A — api/engine/portfolio.ts (new file, full source)

The portfolio engine: auto-execute preference (fail-closed), fill bookkeeping into the positions ledger, order history, position marks, and P&L aggregation.

api/engine/portfolio.ts — create as a new file

```
import { and, desc, eq, ne } from "drizzle-orm";
import { getDb } from "../queries/connection";
import { positions, orderTickets, users, type OrderTicket } from "@db/schema";
import { marketDataService } from "../marketdata/service";

/**
 * PORTFOLIO — order history, positions, and P&L.
 *
 * Every fill (manual CONFIRM or auto-execute) lands here via recordFill:
 *   BUY  → open a position (or average into an existing open one)
 *   SELL → close matching open position(s), booking realized P&L
 *
 * Unrealized P&L marks open positions to the live feed's last price when
 * available, and honestly reports when a mark is unavailable (no feed)
 * instead of inventing a price.
 */

/* ---------- auto-execute preference ---------- */

export async function isAutoExecuteEnabled(userId: number): Promise<boolean> {
  try {
    const db = getDb();
    const [u] = await db.select({ autoExecute: users.autoExecute }).from(users).where(eq(users.id,
userId)).limit(1);
    return u?.autoExecute === true;
  } catch {
    return false; // fail CLOSED — if we can't read the flag, confirmation is required
  }
}

export async function setAutoExecute(userId: number, enabled: boolean): Promise<{ enabled: boolean }> {
  const db = getDb();
  await db.update(users).set({ autoExecute: enabled }).where(eq(users.id, userId));
  return { enabled };
}

/* ---------- fill recording ---------- */

export async function recordFill(userId: number, ticket: OrderTicket, fillPrice: number, fillQty: number):
Promise<void> {
  if (!(fillPrice > 0) || fillQty <= 0) return;
  const db = getDb();
  try {
    if (ticket.side === "BUY") {
      const [open] = await db
        .select()
        .from(positions)
        .where(and(eq(positions.userId, userId), eq(positions.symbol, ticket.symbol), eq(positions.status,
"OPEN")))
        .limit(1);
      if (open) {
        const totalQty = open.quantity + fillQty;
        const newAvg = (Number(open.avgEntry) * open.quantity + fillPrice * fillQty) / totalQty;
        await db.update(positions).set({ quantity: totalQty, avgEntry: newAvg.toFixed(4)
}).where(eq(positions.id, open.id));
      } else {
        await db.insert(positions).values({
          userId,
```

```
          symbol: ticket.symbol,
          quantity: fillQty,
          avgEntry: String(fillPrice),
          sourceTicketId: ticket.ticketId,
          status: "OPEN",
        });
      }
    } else {
      // SELL — close against the oldest open position for the symbol
      const [open] = await db
        .select()
        .from(positions)
        .where(and(eq(positions.userId, userId), eq(positions.symbol, ticket.symbol), eq(positions.status,
"OPEN")))
        .limit(1);
      if (!open) return; // sell without a tracked position — nothing to match (manual external position)
      const closingQty = Math.min(fillQty, open.quantity);
      const realized = (fillPrice - Number(open.avgEntry)) * closingQty;
      if (closingQty >= open.quantity) {
        await db.update(positions).set({
          status: "CLOSED",
          closedAt: new Date(),
          exitPrice: String(fillPrice),
          realizedPnl: realized.toFixed(2),
        }).where(eq(positions.id, open.id));
      } else {
        await db.update(positions).set({ quantity: open.quantity - closingQty }).where(eq(positions.id,
open.id));
        await db.insert(positions).values({
          userId,
          symbol: open.symbol,
          quantity: closingQty,
          avgEntry: open.avgEntry,
          sourceTicketId: ticket.ticketId,
          status: "CLOSED",
          closedAt: new Date(),
          exitPrice: String(fillPrice),
          realizedPnl: realized.toFixed(2),
        });
      }
    }
  } catch {
    /* position tracking must never break execution */
  }
}

/* ---------- order history ---------- */

export interface OrderHistoryRow {
  ticketId: string;
  symbol: string;
  side: string;
  quantity: number;
  orderType: string;
  state: string;
  autoExecuted: boolean;
  entry: number | null;
  stop: number | null;
  target: number | null;
  filledQuantity: number | null;
  averageFillPrice: number | null;
  createdAt: Date;
  submittedAt: Date | null;
  lastMessage: string | null;
}

export async function listOrders(userId: number, limit = 50): Promise<OrderHistoryRow[]> {
  const db = getDb();
```

```
  const rows = await db
    .select()
    .from(orderTickets)
    .where(and(eq(orderTickets.userId, userId), ne(orderTickets.state, "CREATED")))
    .orderBy(desc(orderTickets.id))
    .limit(Math.min(limit, 200));
  return rows.map((t) => ({
    ticketId: t.ticketId,
    symbol: t.symbol,
    side: t.side,
    quantity: t.quantity,
    orderType: t.orderType,
    state: t.state,
    autoExecuted: t.autoExecuted === true,
    entry: t.entry !== null ? Number(t.entry) : null,
    stop: t.stop !== null ? Number(t.stop) : null,
    target: t.target !== null ? Number(t.target) : null,
    filledQuantity: t.filledQuantity ?? null,
    averageFillPrice: t.averageFillPrice !== null ? Number(t.averageFillPrice) : null,
    createdAt: t.createdAt,
    submittedAt: t.submittedAt ?? null,
    lastMessage: t.lastMessage ?? null,
  }));
}

/* ---------- positions & P&L ---------- */

export interface PositionRow {
  id: number;
  symbol: string;
  quantity: number;
  avgEntry: number;
  status: string;
  openedAt: Date;
  closedAt: Date | null;
  exitPrice: number | null;
  realizedPnl: number | null;
  mark: number | null; // live last price when the feed has one
  unrealizedPnl: number | null; // null when no mark available
}

function toPositionRow(p: typeof positions.$inferSelect): PositionRow {
  const feed = marketDataService.get(p.symbol);
  const mark = feed?.indicators?.last ?? null;
  const isOpen = p.status === "OPEN";
  return {
    id: Number(p.id),
    symbol: p.symbol,
    quantity: p.quantity,
    avgEntry: Number(p.avgEntry),
    status: p.status,
    openedAt: p.openedAt,
    closedAt: p.closedAt ?? null,
    exitPrice: p.exitPrice !== null ? Number(p.exitPrice) : null,
    realizedPnl: p.realizedPnl !== null ? Number(p.realizedPnl) : null,
    mark,
    unrealizedPnl: isOpen && mark !== null ? +((mark - Number(p.avgEntry)) * p.quantity).toFixed(2) :
null,
  };
}

export async function listPositions(userId: number, limit = 50): Promise<PositionRow[]> {
  const db = getDb();
  const rows = await db.select().from(positions).where(eq(positions.userId,
userId)).orderBy(desc(positions.id)).limit(Math.min(limit, 200));
  return rows.map(toPositionRow);
}
```

```
export interface PnlSummary {
  realizedTotal: number;
  unrealizedTotal: number;
  unmarkedPositions: number; // open positions with no live mark
  openCount: number;
  closedCount: number;
  winCount: number;
  lossCount: number;
  note: string;
}

export async function getPnlSummary(userId: number): Promise<PnlSummary> {
  const rows = await listPositions(userId, 200);
  const closed = rows.filter((r) => r.status === "CLOSED");
  const open = rows.filter((r) => r.status === "OPEN");
  const realizedTotal = +closed.reduce((a, r) => a + (r.realizedPnl ?? 0), 0).toFixed(2);
  const unrealizedTotal = +open.reduce((a, r) => a + (r.unrealizedPnl ?? 0), 0).toFixed(2);
  const unmarkedPositions = open.filter((r) => r.unrealizedPnl === null).length;
  return {
    realizedTotal,
    unrealizedTotal,
    unmarkedPositions,
    openCount: open.length,
    closedCount: closed.length,
    winCount: closed.filter((r) => (r.realizedPnl ?? 0) > 0).length,
    lossCount: closed.filter((r) => (r.realizedPnl ?? 0) <= 0).length,
    note: unmarkedPositions > 0
      ? `${unmarkedPositions} open position(s) have no live mark — unrealized P&L excludes them (track the
symbol in the data feed).`
      : "All open positions marked to live feed prices.",
  };
}
```

Appendix B — src/components/OrdersPanel.tsx (new file, full source)

The Orders & P&L panel: AUTO-EXECUTE toggle with warning banner, P&L summary cards, open positions with live marks, and order history with AUTO badges. Mount it on the Autonomous page (Appendix J).

src/components/OrdersPanel.tsx — create as a new file

```
import { trpc } from '@/providers/trpc';
import { Badge } from '@/pages/app/owner/shared';
import { Zap, ZapOff, RefreshCw, TrendingUp, TrendingDown } from 'lucide-react';

interface OrderRow {
  ticketId: string; symbol: string; side: string; quantity: number; orderType: string;
  state: string; autoExecuted: boolean; entry: number | null; stop: number | null;
  target: number | null; filledQuantity: number | null; averageFillPrice: number | null;
  createdAt: string | Date; lastMessage: string | null;
}
interface PositionRow {
  id: number; symbol: string; quantity: number; avgEntry: number; status: string;
  mark: number | null; unrealizedPnl: number | null; realizedPnl: number | null;
  openedAt: string | Date;
}

function stateTone(s: string): 'teal' | 'amber' | 'red' | 'slate' | 'sky' {
  if (s === 'FILLED') return 'teal';
  if (s === 'WORKING' || s === 'SUBMITTING' || s === 'CONFIRMED') return 'sky';
  if (s === 'FAILED' || s === 'REJECTED') return 'red';
  if (s === 'EXPIRED') return 'amber';
  return 'slate';
}

export default function OrdersPanel() {
  const utils = trpc.useUtils();
  const { data: pnl, refetch } = trpc.engine.pnl.useQuery(undefined, { refetchInterval: 15000 });
  const { data: positions } = trpc.engine.positions.useQuery(undefined, { refetchInterval: 15000 });
  const { data: orders } = trpc.engine.orders.useQuery(undefined, { refetchInterval: 15000 });
  const { data: autoExec } = trpc.engine.getAutoExecute.useQuery(undefined);
  const setAuto = trpc.engine.setAutoExecute.useMutation({
    onSuccess: () => utils.engine.getAutoExecute.invalidate(),
  });

  const autoOn = autoExec?.enabled === true;
  const openPositions = (positions ?? []).filter((p: PositionRow) => p.status === 'OPEN');

  return (
    <section className="glass rounded-2xl p-6">
      <div className="mb-4 flex flex-wrap items-center justify-between gap-3">
        <div>
          <h2 className="font-display text-lg font-bold text-slate-900">Orders &amp; P&amp;L</h2>
          <p className="text-xs text-slate-500">Order history, open positions, and the auto-execute switch
for autonomous proposals.</p>
        </div>
        <div className="flex items-center gap-3">
          <button onClick={() => refetch()} className="inline-flex items-center gap-1.5 text-xs font-
semibold text-slate-500 hover:text-slate-700">
            <RefreshCw className="h-3.5 w-3.5" /> Refresh
          </button>
          <button
            onClick={() => setAuto.mutate({ enabled: !autoOn })}
            disabled={setAuto.isPending}
            className={`inline-flex items-center gap-2 rounded-xl px-4 py-2 text-sm font-semibold
transition ${
              autoOn
```

```
                ? 'bg-amber-500/15 text-amber-700 border border-amber-500/30 hover:bg-amber-500/25'
                : 'bg-slate-900/[0.05] text-slate-600 border border-slate-900/10 hover:bg-slate-900/10'
            }`}
          >
            {autoOn ? <Zap className="h-4 w-4" /> : <ZapOff className="h-4 w-4" />}
            Auto-execute {autoOn ? 'ON' : 'OFF'}
          </button>
        </div>
      </div>

      {autoOn && (
        <div className="mb-4 rounded-xl border border-amber-500/30 bg-amber-500/10 px-4 py-2.5 text-xs
text-amber-700">
          <b>Auto-execute is ON.</b> Autonomous engine proposals will buy <b>without</b> asking for your
CONFIRM string — every fill still creates a ticket, an alert, and an order-history entry marked AUTO.
Natural-language (chat) trades still always require confirmation. Turn this off anytime to return to
confirm-first mode.
        </div>
      )}

      {/* P&L summary */}
      <div className="mb-5 grid gap-3 sm:grid-cols-4">
        <div className="rounded-xl border border-slate-900/10 p-4">
          <p className="text-[11px] uppercase tracking-wide text-slate-500">Realized P&amp;L</p>
          <p className={`mt-1 text-xl font-bold ${(pnl?.realizedTotal ?? 0) >= 0 ? 'text-teal-600' :
'text-red-600'}`}>
            {(pnl?.realizedTotal ?? 0) >= 0 ? '+' : ''}${(pnl?.realizedTotal ?? 0).toLocaleString()}
          </p>
          <p className="text-[11px] text-slate-500">{pnl?.closedCount ?? 0} closed · {pnl?.winCount ?? 0}W
/ {pnl?.lossCount ?? 0}L</p>
        </div>
        <div className="rounded-xl border border-slate-900/10 p-4">
          <p className="text-[11px] uppercase tracking-wide text-slate-500">Unrealized P&amp;L</p>
          <p className={`mt-1 text-xl font-bold ${(pnl?.unrealizedTotal ?? 0) >= 0 ? 'text-teal-600' :
'text-red-600'}`}>
            {(pnl?.unrealizedTotal ?? 0) >= 0 ? '+' : ''}${(pnl?.unrealizedTotal ?? 0).toLocaleString()}
          </p>
          <p className="text-[11px] text-slate-500">{(pnl?.unmarkedPositions ?? 0) > 0 ?
`${pnl?.unmarkedPositions} unmarked` : 'marked to feed'}</p>
        </div>
        <div className="rounded-xl border border-slate-900/10 p-4">
          <p className="text-[11px] uppercase tracking-wide text-slate-500">Open positions</p>
          <p className="mt-1 text-xl font-bold text-slate-900">{pnl?.openCount ?? 0}</p>
          <p className="text-[11px] text-slate-500">{openPositions.map((p: PositionRow) =>
p.symbol).slice(0, 4).join(', ') || '—'}</p>
        </div>
        <div className="rounded-xl border border-slate-900/10 p-4">
          <p className="text-[11px] uppercase tracking-wide text-slate-500">Execution mode</p>
          <p className={`mt-1 text-xl font-bold ${autoOn ? 'text-amber-600' : 'text-slate-900'}`}>{autoOn
? 'AUTO' : 'CONFIRM'}</p>
          <p className="text-[11px] text-slate-500">{autoOn ? 'no confirmation required' : 'CONFIRM string
required'}</p>
        </div>
      </div>

      {/* open positions */}
      {openPositions.length > 0 && (
        <div className="mb-5">
          <h3 className="mb-2 text-sm font-semibold text-slate-800">Open positions</h3>
          <div className="overflow-x-auto">
            <table className="w-full text-sm">
              <thead>
                <tr className="border-b border-slate-900/5 text-left text-[11px] uppercase tracking-wide
text-slate-500">
                  <th className="px-3 py-2">Symbol</th><th className="px-3 py-2">Qty</th><th
className="px-3 py-2">Avg entry</th>
                  <th className="px-3 py-2">Mark</th><th className="px-3 py-2">Unrealized</th>
                </tr>
```

```
              </thead>
              <tbody>
                {openPositions.map((p: PositionRow) => (
                  <tr key={p.id} className="border-b border-slate-900/[0.04] last:border-0">
                    <td className="px-3 py-2 font-semibold text-slate-800">{p.symbol}</td>
                    <td className="px-3 py-2 text-slate-600">{p.quantity}</td>
                    <td className="px-3 py-2 text-slate-600">${p.avgEntry.toFixed(2)}</td>
                    <td className="px-3 py-2 text-slate-600">{p.mark !== null ? `$${p.mark.toFixed(2)}` :
'no mark'}</td>
                    <td className={`px-3 py-2 font-semibold ${(p.unrealizedPnl ?? 0) >= 0 ? 'text-teal-
600' : 'text-red-600'}`}>
                      {p.unrealizedPnl !== null ? (
                        <span className="inline-flex items-center gap-1">
                          {p.unrealizedPnl >= 0 ? <TrendingUp className="h-3.5 w-3.5" /> : <TrendingDown
className="h-3.5 w-3.5" />}
                          {p.unrealizedPnl >= 0 ? '+' : ''}${p.unrealizedPnl.toFixed(2)}
                        </span>
                      ) : '—'}
                    </td>
                  </tr>
                ))}
              </tbody>
            </table>
          </div>
        </div>
      )}

      {/* order history */}
      <h3 className="mb-2 text-sm font-semibold text-slate-800">Order history</h3>
      {!orders || orders.length === 0 ? (
        <p className="py-6 text-center text-sm text-slate-500">No orders yet — confirmed and auto-executed
tickets will appear here.</p>
      ) : (
        <div className="overflow-x-auto">
          <table className="w-full text-sm">
            <thead>
              <tr className="border-b border-slate-900/5 text-left text-[11px] uppercase tracking-wide
text-slate-500">
                <th className="px-3 py-2">Ticket</th><th className="px-3 py-2">Symbol</th><th
className="px-3 py-2">Side</th>
                <th className="px-3 py-2">Qty</th><th className="px-3 py-2">State</th><th className="px-3
py-2">Fill</th>
                <th className="px-3 py-2">Stop / Target</th><th className="px-3 py-2">When</th>
              </tr>
            </thead>
            <tbody>
              {(orders as OrderRow[]).map((o) => (
                <tr key={o.ticketId} className="border-b border-slate-900/[0.04] last:border-0">
                  <td className="px-3 py-2 font-mono text-xs text-slate-600">
                    {o.ticketId}
                    {o.autoExecuted && <Badge tone="amber">AUTO</Badge>}
                  </td>
                  <td className="px-3 py-2 font-semibold text-slate-800">{o.symbol}</td>
                  <td className="px-3 py-2 text-slate-600">{o.side}</td>
                  <td className="px-3 py-2 text-slate-600">{o.quantity}</td>
                  <td className="px-3 py-2"><Badge tone={stateTone(o.state)}>{o.state}</Badge></td>
                  <td className="px-3 py-2 text-slate-600">
                    {o.averageFillPrice !== null ? `${o.filledQuantity ?? o.quantity} @
$${o.averageFillPrice.toFixed(2)}` : '—'}
                  </td>
                  <td className="px-3 py-2 text-xs text-slate-500">
                    {o.stop !== null ? `$${o.stop.toFixed(2)}` : '—'} / {o.target !== null ?
`$${o.target.toFixed(2)}` : '—'}
                  </td>
                  <td className="px-3 py-2 text-xs text-slate-500">{new
Date(o.createdAt).toLocaleString()}</td>
                </tr>
              ))}
```

```
            </tbody>
          </table>
        </div>
      )}
    </section>
  );
}
```

Appendix C — db/schema.ts edits

Three additions. (1) Inside the `users` table definition, after `govRole`:

users table — new column

```
  govRole: varchar("govRole", { length: 32 }).notNull().default("USER"),
  /** When true, AUTONOMOUS-origin engine proposals execute without the CONFIRM string. Default off. */
  autoExecute: boolean("autoExecute").notNull().default(false),
  createdAt: timestamp("createdAt").defaultNow().notNull(),
  updatedAt: timestamp("updatedAt")
```

(2) Inside the `orderTickets` table definition, after `accountId`:

orderTickets table — new column

```
  rr: decimal("rr", { precision: 6, scale: 2 }),
  /** True when executed via the auto-execute path (no CONFIRM string) — audit marker. */
  autoExecuted: boolean("autoExecuted").notNull().default(false),
  idempotencyKey: varchar("idempotencyKey", { length: 64 }).notNull(),
  brokerOrderId: varchar("brokerOrderId", { length: 64 }),
```

(3) Append the positions table (and its type export) at the end of the schema:

positions table — new

```
/* ---------- Portfolio: positions opened from fills, for P&L ---------- */

export const positions = mysqlTable("positions", {
  id: serial("id").primaryKey(),
  userId: bigint("userId", { mode: "number", unsigned: true }).notNull(),
  symbol: varchar("symbol", { length: 16 }).notNull(),
  quantity: int("quantity").notNull(),
  avgEntry: decimal("avgEntry", { precision: 12, scale: 4 }).notNull(),
  sourceTicketId: varchar("sourceTicketId", { length: 64 }),
  status: mysqlEnum("status", ["OPEN", "CLOSED"]).notNull().default("OPEN"),
  openedAt: timestamp("openedAt").defaultNow().notNull(),
  closedAt: timestamp("closedAt"),
  exitPrice: decimal("exitPrice", { precision: 12, scale: 4 }),
  realizedPnl: decimal("realizedPnl", { precision: 12, scale: 2 }),
  createdAt: timestamp("createdAt").defaultNow().notNull(),
});
export type Position
```

Appendix D — api/lib/ensure-schema.ts edits

Append the positions CREATE TABLE to `STATEMENTS`, the two ALTER statements to `COLUMN_STATEMENTS` (each guarded the same way as the existing column adds), and add `"positions"` to `USER_ID_TABLES`:

api/lib/ensure-schema.ts — added statements

```
  `CREATE TABLE IF NOT EXISTS positions (
    id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
    userId BIGINT UNSIGNED NOT NULL,
    symbol VARCHAR(16) NOT NULL,
    quantity INT NOT NULL,
    avgEntry DECIMAL(12,4) NOT NULL,
    sourceTicketId VARCHAR(64),
    status ENUM('OPEN','CLOSED') NOT NULL DEFAULT 'OPEN',
    openedAt TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    closedAt TIMESTAMP NULL,
    exitPrice DECIMAL(12,4),
    realizedPnl DECIMAL(12,2),
    createdAt TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    INDEX idx_positions_user (userId, status)
  )`,
];

/** Column-level guards for tables that predate newer features. */
const COLUMN_STATEMENTS: string[] = [
  // RBAC access level for the governance boundary (USER by default)
  `ALTER TABLE users ADD COLUMN govRole VARCHAR(32) NOT NULL DEFAULT 'USER' AFTER role`,
  // Auto-execute preference (default OFF — confirmation still required unless enabled)
  `ALTER TABLE users ADD COLUMN autoExecute TINYINT(1) NOT NULL DEFAULT 0 AFTER govRole`,
  // Audit marker on tickets executed via the auto path
  `ALTER TABLE order_tickets ADD COLUMN autoExecuted TINYINT(1) NOT NULL DEFAULT 0 AFTER accountId`,
];

/** Tables that carry a userId foreign reference, for duplicate-user merges. */
const USER_ID_TABLES = ["strategies", "broker_accounts", "alerts", "run_events", "run_sessions",
"ai_limits", "support_tickets", "chat_messages", "trade_outcomes", "positions"];
```

Appendix E — api/queries/tickets.ts edits

(1) Import the portfolio hook:

import (top of file)

```
import { createHash } from "crypto";
import { and, desc, eq, sql } from "drizzle-orm";
import { getDb } from "./connection";
import { alerts, orderTickets, type OrderTicket } from "@db/schema";
import { resolveBroker, type BrokerCode } from "../brokers/registry";
import type { OrderIntent } from "../brokers/types";
import { loadActivePackage, validateConfirmationString } from "../governance/runtime";
import { recordFill } from "../engine/portfolio";

/**
 * ORDER TICKET SERVICE
 *
```

(2) Inside `confirmTicket`, immediately after the FILLED/WORKING transition — book the fill:

confirmTicket — fill hook

```
    brokerOrderId: ack.brokerOrderId,
    filledQuantity: ack.filledQuantity ?? null,
    averageFillPrice: ack.averagePrice !== undefined ? String(ack.averagePrice) : null,
    lastMessage: ack.message ?? null,
  });
  if (filled) {
    await recordFill(userId, ticket, Number(ack.averagePrice ?? ticket.entry ?? 0), ack.filledQuantity ??
ticket.quantity).catch(() => undefined);
  }

  // Execution alert + resolve the confirmation alert.
  const db = getDb();
  await db.insert(alerts).values({
```

(3) Add `autoExecuteTicket` — the confirm path minus the confirmation string, with the louder audit trail:

autoExecuteTicket — new function

```
/**
 * AUTO-EXECUTE — the explicit user-opt-in path that skips the CONFIRM string.
 *
 * Hard rules:
 * - Only reachable when the USER has enabled auto-execute (checked by the
 *   caller — monitor.ts — against the users table, default OFF).
 * - Everything else is identical to a confirmed order: same ticket artifact,
 *   same broker adapter, same idempotency key, same UNKNOWN-is-dead-end rule.
 * - Every auto-execution is flagged (autoExecuted) and alerted as
 *   "AUTO-EXECUTED" — the audit trail is louder, not quieter, than manual.
 * - Only AUTONOMOUS engine proposals use this path. Natural-language
 *   (INTELLIGENCE) and MANUAL tickets always require the CONFIRM string.
 */
export async function autoExecuteTicket(userId: number, ticketIdRaw: string): Promise<{ ok: boolean;
reasonCode: string; ticket?: PublicTicket; message: string }> {
  const ticketId = ticketIdRaw.toUpperCase();
  await expireStaleTickets(userId);
  const ticket = await findTicket(userId, ticketId);
  if (!ticket) return { ok: false, reasonCode: "UNKNOWN_TICKET", message: `No ticket ${ticketId} exists
for this account.` };
```

```
  if (ticket.state !== "READY_FOR_CONFIRMATION") {
    return { ok: false, reasonCode: "TICKET_NOT_EXECUTABLE", message: `Ticket is ${ticket.state} — only
READY_FOR_CONFIRMATION tickets can execute.` };
  }
  if (new Date(ticket.expiresAt).getTime() < Date.now()) {
    await transition(ticket.ticketId, "EXPIRED");
    return { ok: false, reasonCode: "WINDOW_EXPIRED", message: "The execution window has expired. The
ticket is dead and will never be routed." };
  }

  await transition(ticket.ticketId, "CONFIRMED", { confirmedAt: new Date(), autoExecuted: true });

  const { adapter, effective } = resolveBroker(ticket.broker as BrokerCode);
  const accountId = ticket.accountId ?? (await adapter.getAccounts())[0]?.accountId ?? "PAPER-001";
  const intent: OrderIntent = {
    symbol: ticket.symbol,
    side: ticket.side,
    quantity: ticket.quantity,
    orderType: ticket.orderType,
    limitPrice: ticket.limitPrice !== null ? Number(ticket.limitPrice) : undefined,
    stopPrice: ticket.stopPrice !== undefined && ticket.stopPrice !== null ? Number(ticket.stopPrice) :
undefined,
    tif: (ticket.tif as OrderIntent["tif"]) ?? "DAY",
    clientOrderId: ticket.idempotencyKey,
  };

  await transition(ticket.ticketId, "SUBMITTING", { accountId, submittedAt: new Date() });
  let ack;
  try {
    ack = await adapter.placeOrder(accountId, intent);
  } catch (e) {
    await transition(ticket.ticketId, "FAILED", { lastMessage: `broker call failed: ${(e as
Error).message.slice(0, 200)}` });
    return { ok: false, reasonCode: "BROKER_UNREACHABLE", message: `Broker call failed: ${(e as
Error).message}` };
  }

  if (ack.status === "UNKNOWN") {
    await transition(ticket.ticketId, "FAILED", { lastMessage: `broker returned UNKNOWN — manual
reconciliation required, no resubmission. ${ack.message ?? ""}` });
    return { ok: false, reasonCode: "BROKER_STATE_UNKNOWN", message: "Broker returned an unresolvable
state. The order was NOT resubmitted. Reconcile with the broker before retrying." };
  }
  if (ack.status === "REJECTED") {
    await transition(ticket.ticketId, "REJECTED", { brokerOrderId: ack.brokerOrderId, lastMessage:
ack.message ?? "broker rejected" });
    return { ok: false, reasonCode: "BROKER_REJECTED", message: `Broker rejected the order: ${ack.message
?? "no reason given"}` };
  }

  const filled = ack.status === "FILLED";
  await transition(ticket.ticketId, filled ? "FILLED" : "WORKING", {
    brokerOrderId: ack.brokerOrderId,
    filledQuantity: ack.filledQuantity ?? null,
    averageFillPrice: ack.averagePrice !== undefined ? String(ack.averagePrice) : null,
    lastMessage: ack.message ?? null,
  });
  if (filled) {
    await recordFill(userId, ticket, Number(ack.averagePrice ?? ticket.entry ?? 0), ack.filledQuantity ??
ticket.quantity).catch(() => undefined);
  }

  const db = getDb();
  await db.insert(alerts).values({
    userId,
    type: "EXECUTION",
    priority: "CRITICAL",
    title: filled ? "AUTO-EXECUTED — Order Filled" : "AUTO-EXECUTED — Order Working",
```

```
    body: filled
      ? `AUTO-EXECUTE was ON · ${ticket.symbol} ${ticket.side} ${ack.filledQuantity ?? ticket.quantity}
filled @ ${ack.averagePrice ?? "market"} · broker order ${ack.brokerOrderId} · via ${effective} · no human
confirmation (user setting)`
      : `AUTO-EXECUTE was ON · ${ticket.symbol} ${ticket.side} ${ticket.quantity} acknowledged
(${ack.brokerOrderId}) · working · via ${effective}`,
    symbol: ticket.symbol,
    state: filled ? "FILLED" : "ACTIVE",
  });
  await db.update(alerts).set({ state: "EXECUTING" }).where(and(eq(alerts.userId, userId),
eq(alerts.state, "AWAITING_CONFIRMATION"), sql`${alerts.body} LIKE ${"%" + ticket.ticketId + "%"}`));

  const [finalTicket] = await db.select().from(orderTickets).where(eq(orderTickets.ticketId,
ticket.ticketId)).limit(1);
  return { ok: true, reasonCode: filled ? "FILLED" : "SUBMITTED", ticket: toPublic(finalTicket), message:
filled ? `Auto-executed: filled ${ack.filledQuantity ?? ticket.quantity} @ ${ack.averagePrice ?? "market"}
via ${effective}.` : `Auto-executed: order acknowledged by ${effective} and working.` };
}
```

Appendix F — api/engine/monitor.ts edit

Import `autoExecuteTicket` alongside `proposeTicket`, and `isAutoExecuteEnabled` from `./portfolio`. Then replace the single detail line after `proposeTicket` with the branch:

evaluateMonitor — after proposeTicket

```
        });
        ticket = result.ticket;
        if (await isAutoExecuteEnabled(userId)) {
          // User opted in: AUTONOMOUS proposals execute without the CONFIRM
          // string. Same ticket artifact, same broker path, louder audit.
          const exec = await autoExecuteTicket(userId, result.ticket.ticketId);
          ticket = exec.ticket ?? ticket;
          detail = exec.ok
            ? `AUTO-EXECUTED → ${exec.message} (${size.qty} sh, risk $${size.dollarRisk}, capped by
${size.cappedBy})`
            : `auto-execute failed (${exec.reasonCode}: ${exec.message}) — ticket
${result.ticket.ticketId} remains staged for manual CONFIRM`;
        } else {
          detail = `CONFIRMED → ticket ${result.ticket.ticketId} staged (${size.qty} sh, risk
$${size.dollarRisk}, capped by ${size.cappedBy}) — awaiting CONFIRM ORDER ${result.ticket.ticketId}`;
        }
      }
    }
  }
```

Appendix G — api/engine/scanner.ts edit

Add the identification interface and function before `scanFeeds`, add `confidence` and `reasons` to the `Candidate` interface, and call `identifyStrategy` inside `scanFeeds` so every candidate carries them:

identifyStrategy — new

```
/**
 * STRATEGY SELF-IDENTIFICATION.
 *
 * Classifies a symbol's current tape into the best-fit strategy config,
 * with a confidence score and an auditable reason list. Used by the
 * scanner, by monitorStart("AUTO"), and exposed to Intelligence — the
 * system names its own strategy and can explain why.
 */
export interface StrategyIdentification {
  strategyId: string;
  label: string;
  confidence: number; // 0–100
  reasons: string[];
}

export function identifyStrategy(f: {
  gapPct: number | null;
  vsVwapPct: number | null;
  rsi14: number | null;
  relativeVolume: number | null;
}): StrategyIdentification {
  const reasons: string[] = [];
  let strategyId = "general_intraday_momentum";
  let confidence = 40;

  const gap = f.gapPct;
  const vsVwap = f.vsVwapPct;
  const rsi = f.rsi14;
  const rvol = f.relativeVolume;

  if (gap !== null && gap <= -MIN_GAP_PCT && (vsVwap ?? 0) < 0) {
    strategyId = "bottom_fish_reversal";
    confidence = 55;
    reasons.push(`down-gap ${gap}% and trading below VWAP — capitulation profile`);
    if (rsi !== null && rsi < 35) { confidence += 20; reasons.push(`RSI ${rsi} oversold — reversal setup
strengthens`); }
    if (rvol !== null && rvol >= MIN_RVOL) { confidence += 10; reasons.push(`relative volume ${rvol}x —
capitulation volume confirms interest`); }
    if (rsi !== null && rsi > 50) { confidence -= 15; reasons.push(`RSI ${rsi} not oversold — reversal
evidence weakens`); }
  } else if ((gap !== null && gap >= MIN_GAP_PCT) || ((vsVwap ?? 0) > 0 && (rvol ?? 0) >= MIN_RVOL)) {
    strategyId = "post_earnings_continuation";
    confidence = 55;
    if (gap !== null && gap >= MIN_GAP_PCT) reasons.push(`up-gap ${gap}% — event-strength profile`);
    if ((vsVwap ?? 0) > 0) reasons.push(`holding ${vsVwap}% above VWAP — buyers in control`);
    if (rsi !== null && rsi >= 55 && rsi <= 70) { confidence += 15; reasons.push(`RSI ${rsi} in the
momentum band — trend room without exhaustion`); }
    if (rvol !== null && rvol >= 2) { confidence += 15; reasons.push(`relative volume ${rvol}x —
institutional-size interest`); }
    if (rsi !== null && rsi > 75) { confidence -= 20; reasons.push(`RSI ${rsi} overbought — chase risk,
confidence reduced`); }
  } else {
    reasons.push("liquid mover without a clear gap/reversal signature — default momentum config");
    if ((vsVwap ?? 0) > 0) { confidence += 10; reasons.push("above VWAP — mild bullish tilt"); }
    if (rsi !== null && rsi < 30) { confidence += 10; reasons.push(`RSI ${rsi} deeply oversold — watch for
reversal upgrade`); }
  }
```

```
  const cfg = BUILTIN_CONFIGS.find((c) => c.strategy_id === strategyId);
  return {
    strategyId,
    label: cfg?.label ?? strategyId,
    confidence: Math.max(5, Math.min(95, confidence)),
    reasons,
  };
}
```

Appendix H — api/engine-router.ts edits

Imports: `identifyStrategy` from `./engine/scanner`; `getPnlSummary, isAutoExecuteEnabled, listOrders, listPositions, setAutoExecute` from `./engine/portfolio`. Then the helper and the route block:

identifyForSymbol helper

```
/** Compute scanner features for one tracked symbol and run strategy self-identification. */
function identifyForSymbol(symbol: string) {
  const feed = marketDataService.get(symbol.toUpperCase());
  if (!feed || !feed.indicators || feed.bars.length < 30) return null;
  const ind = feed.indicators;
  const gapPct = ind.priorDay && ind.priorDay.close > 0 && feed.bars.length > 0
    ? +(((feed.bars[0].o - ind.priorDay.close) / ind.priorDay.close) * 100).toFixed(2)
    : null;
  const vsVwapPct = ind.vwap && ind.last ? +(((ind.last - ind.vwap) / ind.vwap) * 100).toFixed(2) : null;
  // RVOL against the same bar-count window of the prior session
  const todayVol = feed.bars.slice(-ind.barCount).reduce((a, b) => a + b.v, 0);
  const priorWindow = feed.bars.slice(0, Math.max(feed.bars.length - ind.barCount, 0)).slice(-
ind.barCount);
  const priorVol = priorWindow.reduce((a, b) => a + b.v, 0);
  const relativeVolume = priorVol > 0 ? +(todayVol / priorVol).toFixed(2) : null;
  return identifyStrategy({ gapPct, vsVwapPct, rsi14: ind.rsi14, relativeVolume });
}
```

routes — AUTO monitor start, identify, portfolio

```
  /** Start live monitoring: symbol × config → proposals (module 6). strategyId "AUTO" self-identifies
from the current tape. */
  monitorStart: authedQuery.input(strategyInput).mutation(async ({ ctx, input }) => {
    await marketDataService.track(input.symbol); // ensure the feed exists
    if (input.strategyId.toUpperCase() === "AUTO") {
      const identified = identifyForSymbol(input.symbol);
      if (!identified) throw new Error(`Cannot auto-identify a strategy for ${input.symbol.toUpperCase()}
— no feed data yet. Track the symbol and try again.`);
      const started = await startMonitor(ctx.user.id, input.symbol, identified.strategyId);
      return { ...started, identified };
    }
    return startMonitor(ctx.user.id, input.symbol, input.strategyId);
  }),

  /** Self-identify the best-fit strategy for a symbol from its current tape. */
  identify: authedQuery.input(z.object({ symbol: z.string().min(1).max(12) })).query(({ input }) => {
    const identified = identifyForSymbol(input.symbol);
    if (!identified) return { identified: null, detail: "no feed data — track the symbol first" };
    return { identified };
  }),

  monitorStop: authedQuery.input(z.object({ symbol: z.string().min(1).max(12) })).mutation(async ({ ctx,
input }) => ({
    stopped: stopMonitor(ctx.user.id, input.symbol),
  })),

  monitors: authedQuery.query(async ({ ctx }) => listMonitors(ctx.user.id)),

  /** Evaluate a monitored symbol against its freshest bars right now. */
  evaluate: authedQuery.input(z.object({ symbol: z.string().min(1).max(12) })).mutation(async ({ ctx,
input }) => {
    await marketDataService.refresh(input.symbol).catch(() => undefined);
    return evaluateMonitor(ctx.user.id, input.symbol);
  }),
```

```
  /* ---------- portfolio: orders, positions, P&L, auto-execute ---------- */

  /** Order history — every ticket that became an order (confirmed or auto-executed). */
  orders: authedQuery.query(async ({ ctx }) => listOrders(ctx.user.id, 100)),

  /** Positions with live marks and unrealized P&L. */
  positions: authedQuery.query(async ({ ctx }) => listPositions(ctx.user.id, 100)),

  /** P&L summary: realized + unrealized + win/loss counts. */
  pnl: authedQuery.query(async ({ ctx }) => getPnlSummary(ctx.user.id)),

  /** Auto-execute preference (AUTONOMOUS engine proposals only). */
  getAutoExecute: authedQuery.query(async ({ ctx }) => ({ enabled: await isAutoExecuteEnabled(ctx.user.id)
})),

  setAutoExecute: authedQuery
    .input(z.object({ enabled: z.boolean() }))
    .mutation(async ({ ctx, input }) => setAutoExecute(ctx.user.id, input.enabled)),
});
```

Appendix I — src/components/EnginePanel.tsx edits

(1) Strategy dropdown gains the AUTO option:

strategy select — new option

```
          <option value="AUTO">AUTO — self-identify strategy (monitor only)</option>
```

(2) Backtest / Simulate fall back to the first real config when AUTO is selected (AUTO is monitor-only):

AUTO fallback on backtest/simulate

```
        <ActionBtn onClick={() => backtestMut.mutate({ symbol: sym, strategyId: cfg === 'AUTO' ?
(configs?.[0]?.strategyId ?? '') : cfg })} disabled={!ready || backtestMut.isPending} icon={<FlaskConical
className="h-3.5 w-3.5" />} label={backtestMut.isPending ? 'Backtesting…' : 'Backtest'} />
        <ActionBtn onClick={() => simulateMut.mutate({ symbol: sym, strategyId: cfg === 'AUTO' ?
(configs?.[0]?.strategyId ?? '') : cfg })} disabled={!ready || simulateMut.isPending} icon={<Zap
className="h-3.5 w-3.5" />} label={simulateMut.isPending ? 'Simulating…' : 'Simulate variants'} />
```

(3) Scanner candidate cards show the confidence badge:

candidate card — confidence badge

```
                  <p className="font-semibold text-sky-700">{c.suggestedLabel}
                    {typeof c.confidence === 'number' && (
                      <span className={cn('ml-1.5 rounded-full px-1.5 py-0.5 text-[9px] font-bold',
                        c.confidence >= 70 ? 'bg-emerald-100 text-emerald-700' : c.confidence >= 45 ? 'bg-
amber-100 text-amber-700' : 'bg-slate-100 text-slate-500')}>
                        {c.confidence}%
                      </span>
                    )}
                  </p>
```

(4) Live-monitors header copy reflects the new execution mode:

monitors header

```
        <p className="mb-2 text-xs font-bold text-slate-800">Live monitors — confirmation-gated unless
AUTO-EXECUTE is ON (Orders &amp;amp; P&amp;amp;L panel)</p>
```

empty-state copy

```
          <p className="text-[11px] text-slate-500">No monitors running. Start one above — CONFIRMED
setups arrive as tickets requiring CONFIRM ORDER [TICKET_ID] (or execute automatically when AUTO-EXECUTE
is ON).</p>
```

Appendix J — src/pages/app/Autonomous.tsx edit

Import the panel and mount it after `<EnginePanel />`:

import

```
import OrdersPanel from '@/components/OrdersPanel';
```

mount (inside the page layout)

```
      <OrdersPanel />
```

— End of guide. All source in these appendices is verbatim from the verified v2.2 build (typecheck clean, production build passing, boot log: schema ensured · 16 tables).